ArrayStack – Reverse a Sequence

Java Code and Line-by-Line Explanation

1. Java Code

```
public class Solution {

    public static int[] reverseArray(int[] numbers) {

        ArrayStack stack = new ArrayStack(2);

        // Push all numbers onto the stack
        for (int i = 0; i < numbers.length; i++) {
            stack.push(numbers[i]);
        }

        // Create an array for reversed numbers
        int[] reversed = new int[numbers.length];

        // Pop elements into the new array
        for (int i = 0; i < numbers.length; i++) {
            reversed[i] = stack.pop();
        }

        return reversed;
    }

    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        int[] result = reverseArray(numbers);

        for (int num : result) {
            System.out.print(num + " ");
        }
    }
}
```

Note: This code requires the ArrayStack class provided in your course. That class must have a constructor that accepts the initial capacity, as well as the push() and pop() methods. Adjust the constructor or method names if your class implementation uses different names.

2. Line-by-Line Explanation

public class Solution
Creates a class named Solution.

public static int[] reverseArray(int[] numbers)
Defines a method that receives an integer array named numbers and returns a new reversed integer array.

ArrayStack stack = new ArrayStack(2);
Creates a new ArrayStack with an initial capacity of 2, as required by the assignment.

for (int i = 0; i < numbers.length; i++)
This loop visits every element in the input array.

stack.push(numbers[i]);
Adds each number to the top of the stack using the push() method.

int[] reversed = new int[numbers.length];
Creates a new array with the same length as the input array to store the reversed values.

reversed[i] = stack.pop();
Removes the top element from the stack and stores it in the new array. Because a stack uses LIFO, the values come out in reverse order.

return reversed;
Returns the reversed array to the caller.

main()
The main method runs the example: it creates an input array, calls reverseArray(), and prints the result.

3. How the Stack Works

Input: [10, 20, 30, 40, 50]

The push() method adds the numbers in this order: 10, 20, 30, 40, 50. The last number pushed is 50, so it is at the top of the stack.

The pop() method removes the numbers in this order: 50, 40, 30, 20, 10.

Output: [50, 40, 30, 20, 10]

4. Important Terms

Method / Term	Meaning
push()	Adds an element to the top of the stack.
pop()	Removes and returns the top element of the stack.
isEmpty()	Checks whether the stack is empty.
LIFO	Last In, First Out: the last element added is the first one removed.

5. Complexity Analysis

Time Complexity: O(n) – Each element is pushed onto the stack once and popped once.
Space Complexity: O(n) – The stack and the new output array use additional memory proportional to the number of elements.